

Public Service Reporter — Dokumentacja Techniczna Backend

Repozytorium zawiera logiczną i autoryzacyjną część projektu **public-service-reporter** — platformy stworzonej w celu ułatwienia komunikacji między mieszkańcami a urzędami gmin, automatyzującej proces wykrywania, zgłaszania oraz monitorowania problemów w przestrzeni publicznej.

1. Stos Technologiczny (Tech Stack)

- **Framework główny:** Django 5.x / Django REST Framework (DRF)
- **Uwierzytelnianie:** TokenAuthentication (DRF Token Scheme)
- **Baza danych:** Relacyjna (mapowana przez Django ORM)
- **Kontrola dostępu:** Role-Based Access Control (RBAC) z autorskimi klasami uprawnień

2. Architektura i Struktura Katalogów

Projekt backendowy zorganizowany jest w sposób modułowy, izolując logikę uwierzytelniania od modułów biznesowych.

```
backend/
├── apps/
│   ├── service_auth/           # Moduł uwierzytelniania i użytkowników (label: service_auth)
│   │   ├── apps.py             # Konfiguracja aplikacji Django (BigAutoField, label)
│   │   ├── models.py           # Rozszerzony model użytkownika (AbstractUser)
│   │   ├── serializers.py      # Serializery danych i walidacja ról wejściowych
│   │   ├── urls.py             # Mapowanie ścieżek URL dla modułu auth
│   │   └── views.py            # Klasy widoków API (RegisterView, LoginView)
│   └── reports/               # Moduł zgłoszeń obywatelskich
│       └── permissions.py      # Definicje uprawnień obiektowych i globalnych
```

3. Kontrola Dostępu Oparta na Rolach (RBAC)

Aplikacja implementuje trójstopniowy podział ról użytkowników, definiowany bezpośrednio w modelu bazy danych i walidowany na poziomie warstwy prezentacji danych (serializers):

- **standard** — **Mieszkaniec:** Podstawowa rola przypisywana domyślnie. Posiada uprawnienia do tworzenia zgłoszeń oraz przeglądania publicznych problemów.
- **worker** — **Pracownik Gminy / Urzędnik:** Rola o podwyższonych uprawnieniach. Odpowiedzialna za techniczną i administracyjną obsługę zgłoszeń w danym rejonie.
- **admin** — **Administrator Systemu:** Pełny dostęp administracyjny do zarządzania strukturą bazy danych, użytkownikami i konfiguracją globalną.

4. Modele Danych (Data Models)

Model `User` (Rozszerza `AbstractUser`)

Niestandardowy model użytkownika integrujący natywne mechanizmy bezpieczeństwa Django z logiką biznesową systemu ról.

Pole	Typ danych	Właściwości	Opis
<code>id</code>	Integer	Primary Key, Auto Increment	Unikalny identyfikator użytkownika.
<code>username</code>	CharField	Unique, Max length: 150	Nazwa użytkownika wykorzystywana przy logowaniu.
<code>password</code>	CharField	Write Only (Hash)	Bezpieczny skrót hasła generowany przez <code>set_password()</code> .
<code>email</code>	EmailField	Optional	Adres poczty elektronicznej użytkownika.
<code>role</code>	CharField	Choices, Max length: 10	Rola użytkownika w systemie: <code>standard</code> , <code>worker</code> , <code>admin</code> . Domyślnie: <code>standard</code> .
<code>data</code>	TextField	Nullable, Blank	Pole tekstowe przeznaczone na dodatkowe metadane regionalne (np. nazwa przypisanej gminy).

5. Warstwa Bezpieczeństwa i Uprawnień (Permissions)

Zlokalizowana w module `apps.reports.permissions`, kontroluje dostęp do zasobów na poziomie widoków i pojedynczych obiektów:

- `IsOwner` (Uprawnienie obiektowe):** Weryfikuje relację własnościową obiektu. Zwraca `True` wyłącznie, gdy pole użytkownika w encji biznesowej odpowiada użytkownikowi uwierzytelnionemu w żądaniu (`obj.user == request.user`). Zapobiega nieautoryzowanej modyfikacji zgłoszeń przez innych mieszkańców.
- `IsAdmin` (Uprawnienie globalne):** Zezwala na dostęp do chronionego endpointu tylko wtedy, gdy żądający użytkownik przeszedł poprawnie proces uwierzytelnienia oraz jego pole roli przyjmuje wartość `admin` lub `worker`.

6. Dokumentacja API (Endpoints)

Wszystkie punkty końcowe zwracają odpowiedzi w formacie JSON.

Metoda	Endpoint	Opis działania	Wymagane uprawnienia
<code>POST</code>	<code>/api/auth/register/</code>	Rejestracja nowego konta i walidacja roli.	<code>AllowAny</code>
<code>POST</code>	<code>/api/auth/login/</code>	Uwierzytelnienie danych i wydanie tokenu DRF.	<code>AllowAny</code>

Specyfikacja struktur Request/Response

POST /api/auth/register/

Przyjmuje dane strukturalne nowego użytkownika. Szyfruje hasło przed zapisem encji do bazy danych.

Przykładowe ciało żądania (Request Body):

```
{
  "username": "jan_kowalski",
  "password": "SuperSecurePassword123",
  "email": "jan.kowalski@example.com",
  "role": "standard",
  "data": "Gmina Solina"
}
```

Przykładowa odpowiedź (Response 201 Created):

```
{
  "message": "User has been successfully registered (MOCK)",
  "user": {
    "username": "jan_kowalski",
    "role": "standard",
    "data": "Gmina Solina"
  }
}
```

POST /api/auth/login/

Weryfikuje poświadczenia w bazie danych przy użyciu metody `authenticate()`.

Przykładowe ciało żądania (Request Body):

```
{
  "username": "jan_kowalski",
  "password": "SuperSecurePassword123"
}
```

Przykładowa odpowiedź (Response 200 OK):

```
{
  "token": "a1b2c3d4e5f6g7h8i9j0klmnopqrstuvwxyz1234",
  "role": "standard",
  "user_id": 12
}
```

Status Prac i Kamienie Milowe (Backend TODO)

1. **Refaktoryzacja rejestracji:** Usunięcie struktury mockowej (MOCK) z `RegisterView` i zastąpienie jej pełną serializacją utworzonego obiektu bazodanowego.
2. **Moduł weryfikacji:** Implementacja mechanizmu potwierdzania adresów e-mail za pomocą tokenów aktywacyjnych wysyłanych drogą mailową.

3. **Integracja z raportami:** Połączenie uprawnień `IsOwner` oraz `IsAdmin` z widokami CRUD dla zgłoszeń (moduł `reports`).